Namma Space: phone video to a walkable 3D web twin

Round 1 technical abstract · Team Claudus (Team ID TM-60628EED289) · Laksh Pradhwani · Techfest IIT Bombay 2026, Namma Space (Juspay)

1. Summary

We built an automatic pipeline that turns one smartphone walkthrough video of an indoor space into a photorealistic 3D Gaussian-splat model and serves it in a zero-install web viewer with first-person (WASD and mouse) roaming. There is no manual 3D modelling at any stage: camera poses come from structure-from-motion, appearance from gradient-based splat training, and the viewer loads the resulting file directly. Round 1 asks for SOP construction, 3D model building and free roaming; this document covers exactly those, plus how the same pipeline extends to POI search and path planning in Round 2.

Live viewer: https://namma-space-viewer-eosin.vercel.app | Code: the **Code_Repository** folder inside the submission folder (README, scripts, viewer source, sample data).

2. System architecture

Stage	Input → output	What it does and why
1 Capture (SOP)	Phone video → `.mov/.mp4`	Continuous walk, two laps, locked exposure; see section 3.
2 Frame selection	Video → 480 sharp JPEGs	ffmpeg oversamples 3x, converts HDR (HLG/PQ) to SDR with a Hable tone map, then keeps the sharpest frame per time window (variance of the Laplacian). Removes motion blur and keeps colours correct.
3 Camera poses	Frames → COLMAP sparse model	COLMAP 4.2.1 (CUDA): SIFT features (up to 12,000 per frame), sequential plus exhaustive matching, incremental mapping with global bundle adjustment, one shared SIMPLE_RADIAL camera, then undistortion to a pinhole dataset.
4 Splat training	Poses + frames → `.ply`	Brush trains a 3D Gaussian splat on the GPU through Vulkan (30,000 steps, up to 3 million splats, images capped at 1600 px).
5 Clean-up	`.ply` → web `.spz`	Drops translucent and oversized splats and anything outside a robust box around the dense core (floaters), removes the higher-order spherical-harmonic bands, writes the camera-path placement and walking bounds, then packs the model with Spark's own SPZ encoder. 1,386,969 → 892,012 splats, 327 MB → 50 MB (.ply) → 15 MB (.spz).
6 Web viewer	Static files → browser	Spark (WebGL2) on three.js. Pointer-lock mouse look, WASD, Shift to sprint, touch controls on phones, auto walking bounds, an on-screen calibration panel, an fps and splat counter.

3. Standard operating procedure (SOP) for capture

The SOP is what a non-expert follows with a phone alone:

- **Equipment:** any recent smartphone, landscape, 4K at 30 fps if available. No tripod, LiDAR or depth sensor required.
- **Preparation:** turn on all lights, close curtains against glare, move people and pets out, keep doors open so adjoining rooms are visible.
- **Exposure:** lock exposure and focus on a mid-tone surface so brightness does not drift between frames.
- **Route:** one continuous clip. Lap 1 along the walls; lap 2 about a metre inside the room with the lens tilted slightly down and then up. Return to the start point and end on the starting view so the path closes a loop.
- **Motion:** walk slowly, about 0.5 m/s. Rotate the body only while moving; no spinning in place and no fast whip pans, which give narrow baselines and blur. Pause about two seconds at doorways and corners.
- **Coverage target:** every surface seen from at least three positions with 60 to 80 percent overlap between neighbouring frames.
- **Quality gate (automatic):** after pose estimation we report registered frames, 3D points and mean reprojection

error; if fewer than about 60 percent of frames register, the operator is told to recapture the weak section.

> **Honest scope of this submission.** The footage used here is one 85-second 4K HDR pass of a small furnished bedroom, recorded by a collaborator who owns the flat and gave permission. It contains fast pans and plain walls, so it is a hard case and it only partly followed the SOP above (fast pans, no scale bar). We report the resulting numbers as measured in section 5 and plan a clean recapture that follows the full SOP.

4. Technical approach and design decisions

Why Gaussian splatting

The brief asks for photorealism from ordinary photos or video with no manual modelling. Gaussian splats render view-dependent appearance of real surfaces (fabric, wicker, wood grain) in real time in a browser, which meshes from photogrammetry usually lose on thin and glossy structures. Training takes minutes, not hours, on a laptop GPU.

Poses without special hardware

Phone video is the only input. HDR clips are tone-mapped first, because HLG footage otherwise yields washed-out colours and inconsistent photometry. Frames are chosen by sharpness rather than at a fixed interval, which matters for hand-held captures. Feature extraction and matching run on the GPU, so a 480-frame set is matched in minutes. Because a continuous video can break into disconnected components when overlap is low, we match sequentially first and exhaustively second to bridge components.

Training on a laptop GPU

Brush runs on Vulkan, so no CUDA toolkit is needed and it works on a consumer laptop GPU (NVIDIA GeForce RTX 5050 Laptop GPU, 8 GB). The splat count is capped to fit in 8 GB of VRAM.

Web delivery

The viewer is static HTML plus one model file, hosted on a CDN, so a visitor only opens a link. The model is placed from the real camera path: the floor is estimated from the sparse points, scale assumes a 1.4 m phone height (no scale bar was used), the first camera becomes the start pose, and walking is bounded to the filmed path plus a 0.8 m margin so visitors stay where the reconstruction is reliable. Movement is first person with a fixed eye height and delta-time based motion, so speed is the same at any frame rate. All libraries are bundled with the page, so it makes no third-party requests.

Reproducibility

The repository contains one script per stage (`extract_frames.py`, `run_colmap.py` or `run_colmap_gpu.ps1`, `clean_splat.py`, `make_config.py`, `ply_to_spz.mjs`), the viewer, a README with exact commands, and 36 sample frames in `test/` with a test that registers them (35 of 36 registered at 0.55 px in our run).

5. Results on our capture

- **Source video:** 85 s, 3840 x 2160, 30 fps, HDR (HLG) iPhone video.
- **Frames extracted:** 480. **Registered by COLMAP (main model):** 306 of 480 (64 percent), with 105,344 sparse 3D points and a mean reprojection error of 0.68 px.
- **Pose estimation time:** about 26 minutes (GPU features and matching about 10 minutes, CPU mapping about 16 minutes).
- **Splat training time:** 28 minutes (30,000 steps) on the NVIDIA GeForce RTX 5050 Laptop GPU, 8 GB.
- **Final web model:** 892,012 splats, 50 MB as .ply and 15 MB as .spz.
- **Viewer performance:** Measured with an automated test in Chromium on an integrated Intel UHD laptop GPU at 1280 x 720: 41 fps on the local server and 53 fps on the live site with 892,012 splats. The 15 MB model loaded in about 15 seconds from the live site on our connection. WASD movement in all four directions and the walking bounds passed the same test.

Limitations we observed. Only 64 percent of the extracted frames registered (306 of 480): the footage has fast pans and large plain walls, which give COLMAP too little overlap and texture. Areas the phone never saw stay dark instead of showing invented geometry, so the model is a partial room rather than a closed one. Scale is assumed from a 1.4 m phone height because no scale bar was filmed, so distances are approximate. Camera poses at the edge of the capture drifted, so walking is bounded to the well-reconstructed core. A recapture that follows the full SOP (slower pace, second lap, scale bar) should close these gaps.

6. Path to Round 2 and Round 3

- **POI tagging and search:** POIs are named 3D anchors in the same coordinate frame as the splat, stored as JSON, shown as billboards, and searched by name with synonyms. Click-to-name will use a depth raycast against the splat.
- **Non-colliding paths:** a 2D occupancy grid is built by slicing the cleaned splat (and sparse points) between floor and head height; A* with a clearance margin and string-pulling gives smooth walkable routes that avoid walls and furniture. This is the approach used in the splat-navigation literature (occupancy-grid or slice planners over Gaussian maps).
- **On-site capture speed:** the pipeline is scripted end to end, so a new venue is a short chain of commands after filming. On this laptop the measured times were: about 26 minutes (GPU features and matching about 10 minutes, CPU mapping about 16 minutes) for poses and 28 minutes for training, which should fit the Round 3 Day 1 window.
- **AR navigation (bonus):** relocalise the phone against the sparse model, then draw direction arrows along the computed path.

7. Open-source components and licences

- **COLMAP 4.2.1 and pycolmap** (BSD-style licence): features, matching, structure-from-motion, undistortion.
- **Brush 0.3.0** (Apache-2.0): Gaussian splat training on Vulkan.
- **Spark 2.4.0** (MIT): splat rendering and SPZ packing in the browser.
- **three.js 0.180** (MIT): 3D scene and camera.
- **FFmpeg** (LGPL/GPL build): frame extraction and HDR tone mapping.
- **OpenCV** (Apache-2.0), **NumPy** (BSD), **plyfile** (GPL-3.0), **PyMuPDF** (AGPL-3.0): sharpness scoring, splat clean-up, and generating this PDF.

All pipeline scripts and the viewer are our own original work and are in the linked repository.